

Project Title

Air Quality Index Analysis – Exploring Environmental Trends

Objective(Problem statment)

1. Assess Air Quality: Provide a standardized measure of pollutants (like PM2.5, PM10, NO₂, SO₂, CO, O₃) to understand how clean or polluted the air is.
2. Identify Trends: Track changes in air quality across cities
3. Enable Decision-Making: Help policymakers, city planners, and environmental agencies design interventions (e.g., traffic restrictions, industrial regulations).
4. Raise Awareness: Inform the public about safe vs. hazardous air conditions, often through AQI categories (Good, Moderate, Unhealthy, Hazardous).
5. Predictive Analysis: forecast pollution levels.

Outcome

The project provides a clear understanding of how pollutant levels affect both the environment and human health. Through city-wise, it highlights pollution patterns across different regions. Based on these insights, the study suggests practical measures to reduce pollutant levels and improve overall air quality.

Domain

Environment

Dataset Information

Source: UCI ML Repository / Government Portals / Company Internal Dataset / Open Data Portals [with link](#)

raw.githubusercontent.com/Archana-3009/https-www.data.gov.in-catalog-real-time-air-quality-index/refs/heads/main/3b01bcb8-0b14-4abf-b6f2-c1bfd384ba69.csv

Year / Timeline: Data collected during YYYY to YYYY

Dataset Column/features Description:

country: The country where the data is collected (e.g., India).

state: The state name (e.g., Andhra Pradesh, Bihar, Delhi).

city: The city name (e.g., Kadapa, Patna, Guwahati).

station: The specific monitoring station (e.g., “Yerramukkapalli, Kadapa - APPCB”).

last_update: Timestamp of the latest reading (e.g., 15-08-2026 11:00).

latitude & longitude: Geographic coordinates of the station.

pollutant_id: Type of pollutant measured (e.g., PM2.5, PM10, NO₂, SO₂, CO, OZONE, NH₃).

pollutant_min: Minimum recorded value of the pollutant during the period.

pollutant_max: Maximum recorded value of the pollutant during the period.

pollutant_avg: Average concentration of the pollutant during the period

Descriptive Analysis:

The dataset captures air quality across Indian states, showing pollutant levels (PM2.5, PM10, NO2, SO2, CO, Ozone, NH3) with wide variation between urban and rural monitoring stations.

Diagnostic Analysis:

High pollution in cities like Delhi and Patna is linked to traffic and industry, while rural spikes suggest seasonal burning; industrial belts show elevated SO2 and CO, and ozone peaks highlight secondary pollutant formation.

Stages for DA Project

Stage 1 – Problem Definition and Dataset Selection

NOTE : THINGS TO BE COMPLETED

- Define the business problem and expected outcome
- Choose dataset and explain its source, size, and features
- Import libraries, load dataset
- Dataset description (rows, columns, features)
- Initial EDA (head, info, describe, shape, null checks, duplicate check)

Initial EDA

```
import pandas as pd
```

```
AQI = "https://raw.githubusercontent.com/Archana-3009/https-www.data.gov.in-catalog-real-time-air-quality-index/refs/heads/main/data/realtime-air-quality-index.csv"

df = pd.read_csv(AQI)
```

```
pd.set_option('display.max_columns', None) # show all columns
pd.set_option('display.width', 2000) # make console wide enough
pd.set_option('display.colheader_justify', 'left') # align headers neatly
print(df.head())
```

	country	state	city	station		last_update	latitude	longitu
0	India	Andhra Pradesh	Kadapa	Yerramukkapalli, Kadapa - APPCB	15-08-2026 11:00	14.465052	78.8241	
1	India	Andhra Pradesh	Kakinada	Rama Rao Peta, Kakinada - APPCB	15-08-2026 11:00	16.962488	82.2295	
2	India	Andhra Pradesh	Kakinada	Rama Rao Peta, Kakinada - APPCB	15-08-2026 11:00	16.962488	82.2295	
3	India	Andhra Pradesh	Machilipatnam	Srinivas Nagar Colony, Machilipatnam - APPCB	15-08-2026 11:00	16.186555	81.1321	
4	India	Andhra Pradesh	Nellore	Ambedkar Nagar, Nellore - APPCB	15-08-2026 11:00	14.438164	79.9668	

```
# your code here
# If you want more cells, you can add or less cells you can remove, but dont change the structure.
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3514 entries, 0 to 3513
Data columns (total 11 columns):
#   Column              Non-Null Count  Dtype
---  -
0   country              3514 non-null   object
1   state                3514 non-null   object
2   city                 3514 non-null   object
3   station              3514 non-null   object
4   last_update          3514 non-null   object
5   latitude              3514 non-null   float64
6   longitude             3514 non-null   float64
7   pollutant_id         3514 non-null   object
8   pollutant_min        3269 non-null   float64
```

```
9 pollutant_max 3269 non-null float64
10 pollutant_avg 3269 non-null float64
dtypes: float64(5), object(6)
memory usage: 302.1+ KB
```

```
df.shape
```

```
(3514, 11)
```

```
df.isnull().sum()
```

```

      0
country  0
state    0
city     0
station  0
last_update  0
latitude    0
longitude   0
pollutant_id  0
pollutant_min 245
pollutant_max 245
pollutant_avg 245
```

```
dtype: int64
```

```
df.columns
```

```
Index(['country', 'state', 'city', 'station', 'last_update', 'latitude', 'longitude', 'pollutant_id', 'pollutant_min', 'pollutant_max', 'pollutant_avg'], dtype='object')
```

```
df.describe()
```

	latitude	longitude	pollutant_min	pollutant_max	pollutant_avg
count	3514.000000	3514.000000	3269.000000	3269.000000	3269.000000
mean	23.348372	78.552261	16.431019	42.789844	26.654940
std	5.116010	4.832619	17.186871	53.671859	26.091853
min	8.514909	70.776774	0.000000	0.000000	0.000000
25%	19.224333	75.638133	5.000000	11.000000	8.000000
50%	23.939892	77.288278	11.000000	27.000000	19.000000
75%	28.148556	80.518167	23.000000	55.000000	37.000000
max	34.147096	94.636574	258.000000	500.000000	302.000000

```
df['city'].nunique()
```

```
265
```

```
df['pollutant_id'].nunique()
```

```
7
```

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df.sample(5)
```

	country	state	city	station	last_update	latitude	longitude	pollutant_id	pollutant_min	pollutant_max	pollutant_avg
2722	India	Uttar Pradesh	Prayagraj	Jhansi, Prayagraj - UPPCB	15-08-2026 11:00	25.425602	81.917152	SO2	0.0	0.0	0.0
169	India	Delhi	Delhi	Aya Nagar, Delhi - IITM	15-08-2026 11:00	28.470691	77.109936	PM2.5	0.0	77.0	0.0

Double-click (or enter) to edit

Start coding or [generate](#) with AI.

Stage 2 – Data Cleaning and Pre-processing

- Handle missing values (impute or drop)
- Handle duplicates
- column name rename or dropping unwanted columns if required
- Treat outliers if required
- Check skewness and apply transformations
- Convert data types if needed
- Feature transformations (date parts, derived fields if required for analysis)

your code here

```
from sklearn.impute import KNNImputer
```

```
# Select columns for imputation
```

```
cols_to_impute = ['latitude', 'longitude', 'pollutant_min', 'pollutant_max', 'pollutant_avg']
data_for_impute = df[cols_to_impute]
```

```
# Apply KNN Imputer
```

```
imputer = KNNImputer(n_neighbors=3) # use 3 nearest neighbors
imputed_data = imputer.fit_transform(data_for_impute)
```

```
# Put back into DataFrame
```

```
df[cols_to_impute] = pd.DataFrame(imputed_data, columns=cols_to_impute)
```

```
df[['pollutant_min', 'pollutant_max', 'pollutant_avg']] = (
    df[['pollutant_min', 'pollutant_max', 'pollutant_avg']].round(0)
)
```

```
print(df[['pollutant_min', 'pollutant_max', 'pollutant_avg']].isnull().sum())
```

```
pollutant_min    0
pollutant_max    0
pollutant_avg    0
dtype: int64
```

```
df.head()
```

	country	state	city	station	last_update	latitude	longitude	pollutant_id	pollutant_min	pollutant_max	pollutant_avg
0	India	Andhra Pradesh	Kadapa	Yerramukkapalli, Kadapa - APPCB	15-08-2026 11:00	14.465052	78.824187	OZONE	35.0	35.0	35.0
1	India	Andhra Pradesh	Kakinada	Rama Rao Peta, Kakinada - APPCB	15-08-2026 11:00	16.962488	82.229513	NO2	23.0	64.0	23.0
2	India	Andhra Pradesh	Kakinada	Rama Rao Peta, Kakinada - APPCB	15-08-2026 11:00	16.962488	82.229513	NO2	23.0	64.0	23.0

```
# Define categorization function based on average value
def Pollutant_Category (avg):
    if avg <= 50:
        return "Low"
```

```
elif avg <= 100:
    return "Moderate"
elif avg <= 200:
    return "High"
else:
    return "Very High"

# Apply categorization
df['Pollutant_Category'] = df['pollutant_avg'].apply(Pollutant_Category)
```

df.head()

	country	state	city	station	last_update	latitude	longitude	pollutant_id	pollutant_min	pollutant_max
0	India	Andhra Pradesh	Kadapa	Yerramukkapalli, Kadapa - APPCB	15-08-2026 11:00	14.465052	78.824187	OZONE	35.0	37.0
1	India	Andhra Pradesh	Kakinada	Rama Rao Peta, Kakinada - APPCB	15-08-2026 11:00	16.962488	82.229513	NO2	23.0	64.0
2	India	Andhra Pradesh	Kakinada	Rama Rao Peta, Kakinada - APPCB	15-08-2026 11:00	16.962488	82.229513	OZONE	16.0	30.0
3	India	Andhra Pradesh	Machilipatnam	Srinivas Nagar Colony,	2026-08-15 16:18:55	16.186555	81.132190	SO2	4.0	27.0

```
# your code here
# If you want more cells, you can add or less cells you can remove, but dont change the structure.
print(df['Pollutant_Category'].value_counts())
```

```
Pollutant_Category
Low          3041
Moderate     409
High         60
Very High    4
Name: count, dtype: int64
```

```
df['last_update']=pd.to_datetime(df['last_update'],format='%d-%m-%Y %H:%M')
```

```
# Separate date and Time from Last Update
df['Date'] = df['last_update'].dt.date
df['Time'] = df['last_update'].dt.time
```

df.head()

	country	state	city	station	last_update	latitude	longitude	pollutant_id	pollutant_min	pollutant_max
0	India	Andhra Pradesh	Kadapa	Yerramukkapalli, Kadapa - APPCB	2026-08-15 11:00:00	14.465052	78.824187	OZONE	35.0	37.0
1	India	Andhra Pradesh	Kakinada	Rama Rao Peta, Kakinada - APPCB	2026-08-15 11:00:00	16.962488	82.229513	NO2	23.0	64.0
2	India	Andhra Pradesh	Kakinada	Rama Rao Peta, Kakinada - APPCB	2026-08-15 11:00:00	16.962488	82.229513	OZONE	16.0	30.0
3	India	Andhra Pradesh	Machilipatnam	Srinivas Nagar Colony,	2026-08-15 16:18:55	16.186555	81.132190	SO2	4.0	27.0

```
df.drop(columns=['last_update'],inplace=True)
df.drop(columns=['Time'],inplace=True)
```

df.head()

	country	state	city	station	latitude	longitude	pollutant_id	pollutant_min	pollutant_max	pollutant_avg
0	India	Andhra Pradesh	Kadapa	Yerramukkapalli, Kadapa - APPCB	14.465052	78.824187	OZONE	35.0	37.0	36.0
1	India	Andhra Pradesh	Kakinada	Rama Rao Peta, Kakinada - APPCB	16.962488	82.229513	NO2	23.0	64.0	43.5
2	India	Andhra Pradesh	Kakinada	Rama Rao Peta, Kakinada - APPCB	16.962488	82.229513	OZONE	16.0	30.0	23.0
3	India	Andhra Pradesh	Machilipatnam	Srinivas Nagar Colony,	16.186555	81.132190	SO2	4.0	27.0	15.5

```
df["pollutant_avg"]
```

pollutant_avg

0	35.0
1	46.0
2	21.0
3	13.0
4	27.0
...	...
3509	60.0
3510	42.0
3511	15.0
3512	15.0
3513	24.0

3514 rows × 1 columns

dtype: float64

```
# Create new Column - Pollutant Range
df['Pollutant_Range']=df['pollutant_max']-df['pollutant_min']
```

```
#Pollutant Id change
df.rename(columns={'pollutant_id':'Pollutant_Name'},inplace=True)
```

```
# Create Pollutant group based on Pollutant Name
def pollutant_group(name):
    if name in ['PM2.5','PM10']:
        return 'Particulate Matter'
    elif name in ['N02','SO2','CO','O3']:
        return 'Gaseous Pollutant'
    else:
        return 'Other'

df['Pollutant_Group'] = df['Pollutant_Name'].apply(pollutant_group)
```

```
counts = df['Pollutant_Group'].value_counts()
print(counts)
```

```
Pollutant_Group
Gaseous Pollutant    1506
Other                1004
Particulate Matter   1004
Name: count, dtype: int64
```

```
df.groupby(['city','Pollutant_Group']).size().reset_index(name='Count')
```

	city	Pollutant_Group	Count
0	Agartala	Gaseous Pollutant	3
1	Agartala	Other	2
2	Agartala	Particulate Matter	2
3	Agra	Gaseous Pollutant	18
4	Agra	Other	12
...	...	...	...
790	Yadgir	Other	2
791	Yadgir	Particulate Matter	2
792	Yamuna Nagar	Gaseous Pollutant	3
793	Yamuna Nagar	Other	2
794	Yamuna Nagar	Particulate Matter	2

795 rows × 3 columns

```
def health_hazard(Pollutant_Group):
    if Pollutant_Group == 'Particulate Matter':
        return 'Respiratory Hazard'
```

```

elif Pollutant_Group == 'Gaseous Pollutant':
    return 'Cardiovascular Hazard'
elif Pollutant_Group == 'Other':
    return 'Neurological/Toxic Hazard'
else:
    return 'Unknown'

df['Health_Hazard'] = df['Pollutant_Group'].apply(health_hazard)

```

```
df.to_csv('AQI_Cleaned_Data.csv', index=False)
```

Stage 3 – EDA and Visualizations

Univariate Analysis → distribution of single variables (countplot, histogram, boxplot)

Bivariate Analysis → relation between two variables (scatterplot, barplot, correlation heatmap)

Multivariate Analysis → relation among 3+ variables (pairplot, grouped analysis, pivot tables, advanced plots)

Interpretation MUST with every visualization

Focus on business story not just charts

Each chart must be in separate cells.

✓ Note For Kind of Analysis to be done

Type of Analysis

- Descriptive Analysis (summarizing the dataset)
- Diagnostic Analysis (finding reasons behind patterns)
- Predictive Analysis (optional if forecasting required using basic stats)
- Prescriptive Analysis (recommendations for business decisions)

> Analysis about the sales by region

↳ 1 cell hidden

Insights

- point by point insight should be recorded

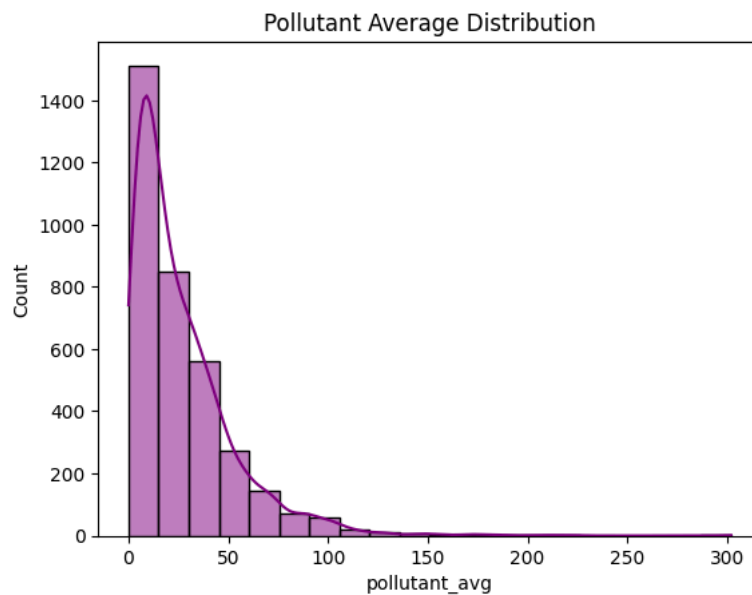
✓ Interpretation of your above chart

```

import matplotlib.pyplot as plt
import seaborn as sns

sns.histplot(df["pollutant_avg"], bins=20, kde=True, color="purple")
plt.title("Pollutant Average Distribution")
plt.figure(figsize=(8,5))
plt.show()

```



<Figure size 800x500 with 0 Axes>

✓ Interpretation of your above chart

Most pollutant averages fall in the lower ranges below 50.

The KDE curve highlights the overall trend, showing a right-skewed distribution.

Occasional extreme spikes Values above 200 - indicate severe pollution events.

This suggests that while air quality is often acceptable, dangerous peaks do occur.

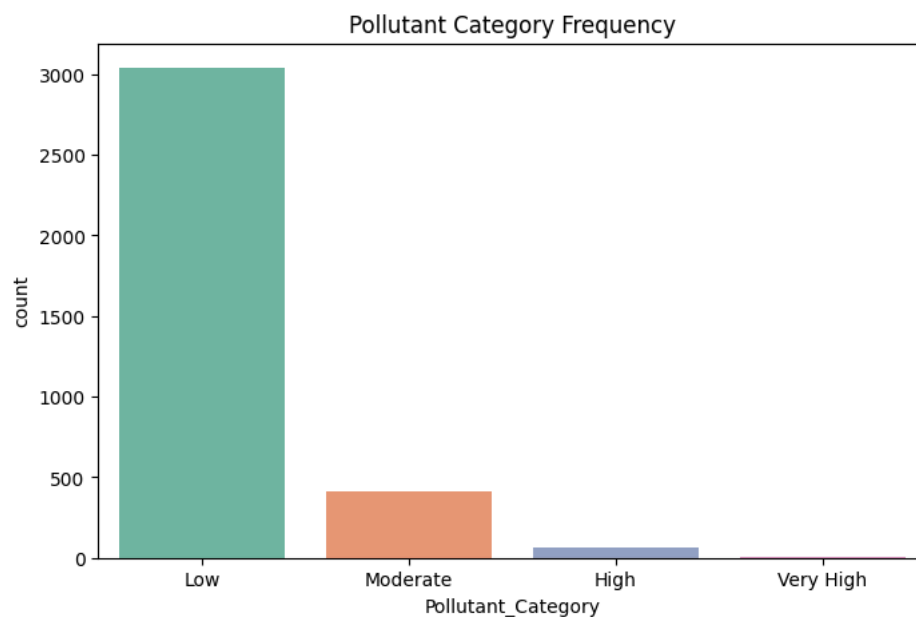
```
# your code here
# If you want more cells, you can add or less cells you can remove, but dont change the structure.

plt.figure(figsize=(8,5))
sns.countplot(x="Pollutant_Category", data=df, palette="Set2")
plt.title("Pollutant Category Frequency")
plt.show()
```

/tmp/ipykernel_4192/1909527457.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

```
sns.countplot(x="Pollutant_Category", data=df, palette="Set2")
```



✓ Interpretation of your above chart

What it shows: Frequency of pollutant categories (Low, Moderate, High, Very High).

Features used: Pollutant_Category.

Interpretation:

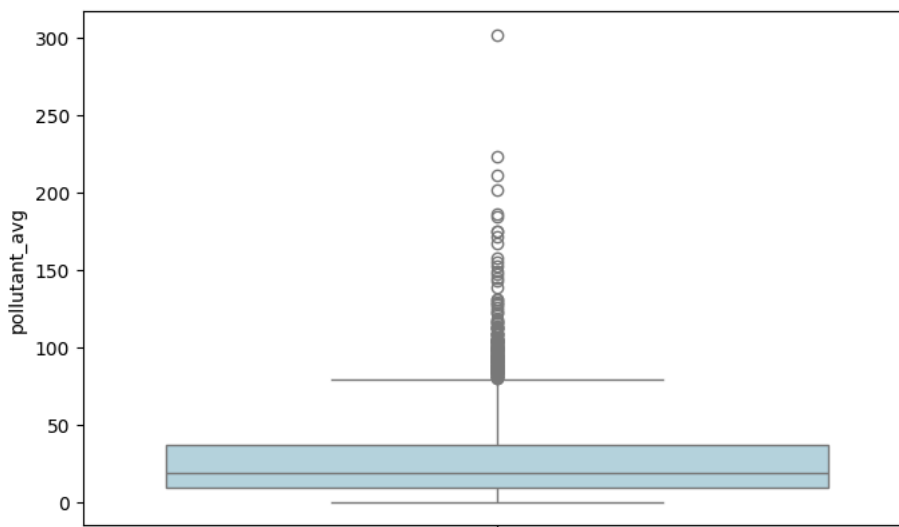
Majority of records are in the Low category ~3000.

Moderate pollution is present but less frequent ~400.

High and Very High categories are rare, showing extreme pollution events are uncommon but critical.

This shows that most areas are safe, but some extreme pollutions are exist.

```
# your code here
# If you want more cells, you can add or less cells you can remove, but dont change the structure.
plt.figure(figsize=(8,5))
sns.boxplot(y="pollutant_avg", data=df, color="lightblue")
plt.title = ("Pollutant Average Boxplot")
plt.show()
```



✓ Interpretation of your above chart

What it shows: Spread and variability of pollutant averages.

Features used: pollutant_avg.

Interpretation:

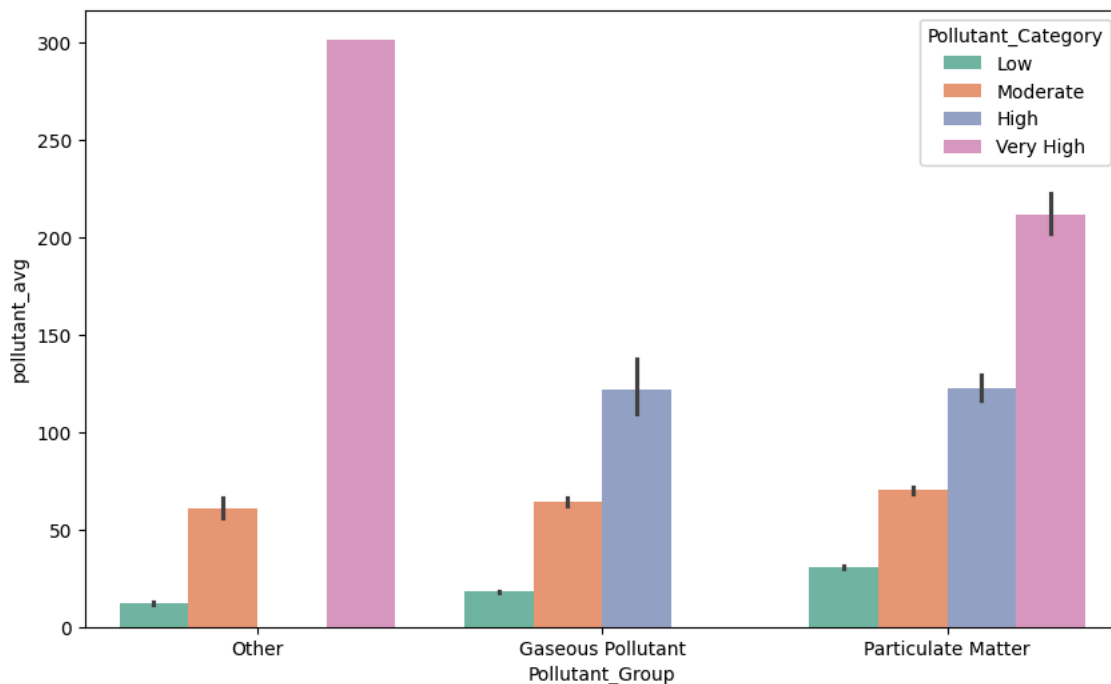
The median lies around 20, showing central tendency.

The interquartile range (IQR) highlights typical pollution levels between ~8 and ~37.

Outliers above 200 represent extreme pollution spikes.

This confirms variability in air quality, with occasional hazardous conditions.

```
plt.figure(figsize=(10,6))
sns.barplot(
    x="Pollutant_Group",
    y="pollutant_avg",
    data=df,
    hue="Pollutant_Category",
    palette="Set2"
)
plt.title("Average Pollutant by Group with Category Breakdown")
plt.show()
```



Interpretation of the above chart

What it shows: Comparison of pollutant groups (Particulate Matter, Gaseous Pollutants, Other) with category breakdown.

Features used: Pollutant_Group, pollutant_avg, Pollutant_Category.

Interpretation:

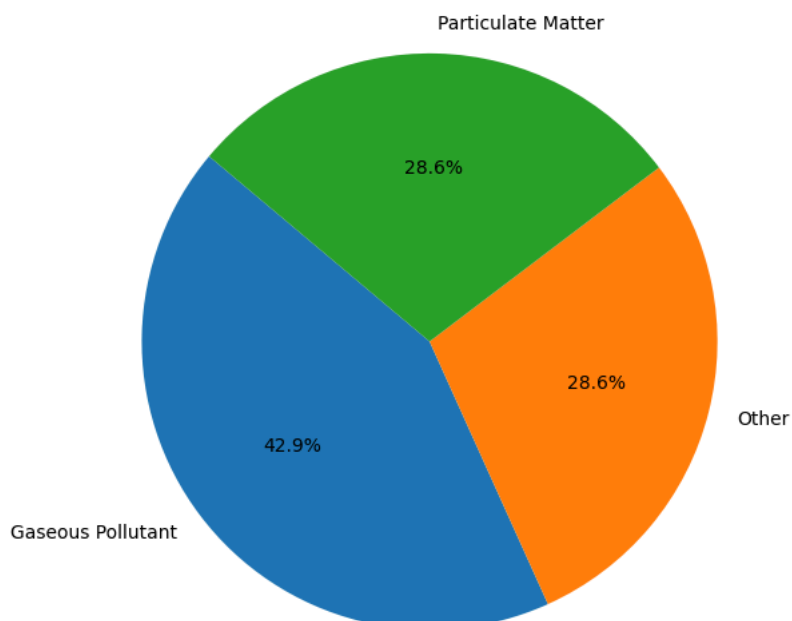
Particulate Matter contributes significantly to pollution, often in Moderate/High ranges.

Gaseous pollutants also show strong averages, linked to traffic and industrial emissions.

Other pollutants occasionally spike, adding toxic hazards.

This chart helps policymakers prioritize interventions (dust control, emission filters, fuel standards).

```
Pollutant_Count = df['Pollutant_Group'].value_counts()
plt.figure(figsize=(7,7))
plt.pie(Pollutant_Count, labels=Pollutant_Count.index, autopct='%1.1f%%', startangle=140)
plt.title("Distribution of Pollutant Groups")
plt.show()
```



Distribution of Pollutant groups The pie chart shows the share of pollutant groups (Particulate Matter, Gaseous Pollutants, Other) in the dataset.

If Gaseous Pollutants occupy the largest slice, it indicates that traffic emissions, industrial gases, and fuel combustion are the dominant contributors to poor air quality. Gaseous pollutant is distributed by 42.9% and contributed major portion.

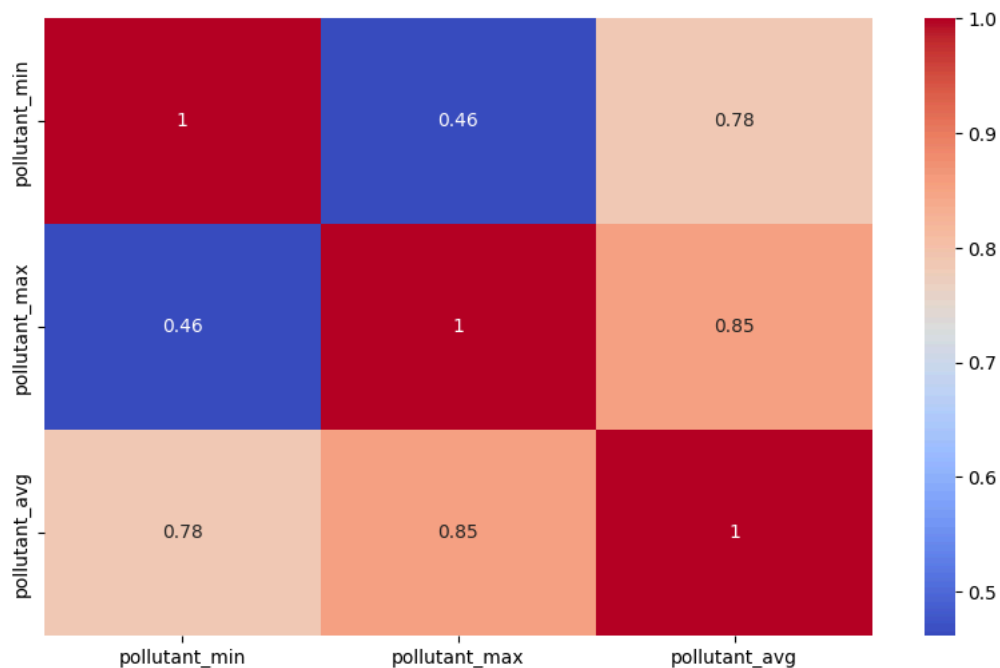
If Particulate Matter (PM2.5, PM10) is significant, it highlights dust, construction, and crop burning as major pollution sources.

The Other pollutants (like NH₃, Pb, Benzene), though smaller in proportion, represent toxic hazards that can affect neurological health.

particulate Matter and other contributed 28.6%. equally polluted groups.

This visualization is meaningful because it summarizes the overall composition of pollution types in one glance, making it easy for policymakers and the public to understand which pollutants are most problematic.

```
plt.figure(figsize=(10,6))
corr = df[["pollutant_min","pollutant_max","pollutant_avg"]].corr()
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Correlation Matrix: Pollutant Metrics")
plt.show()
```

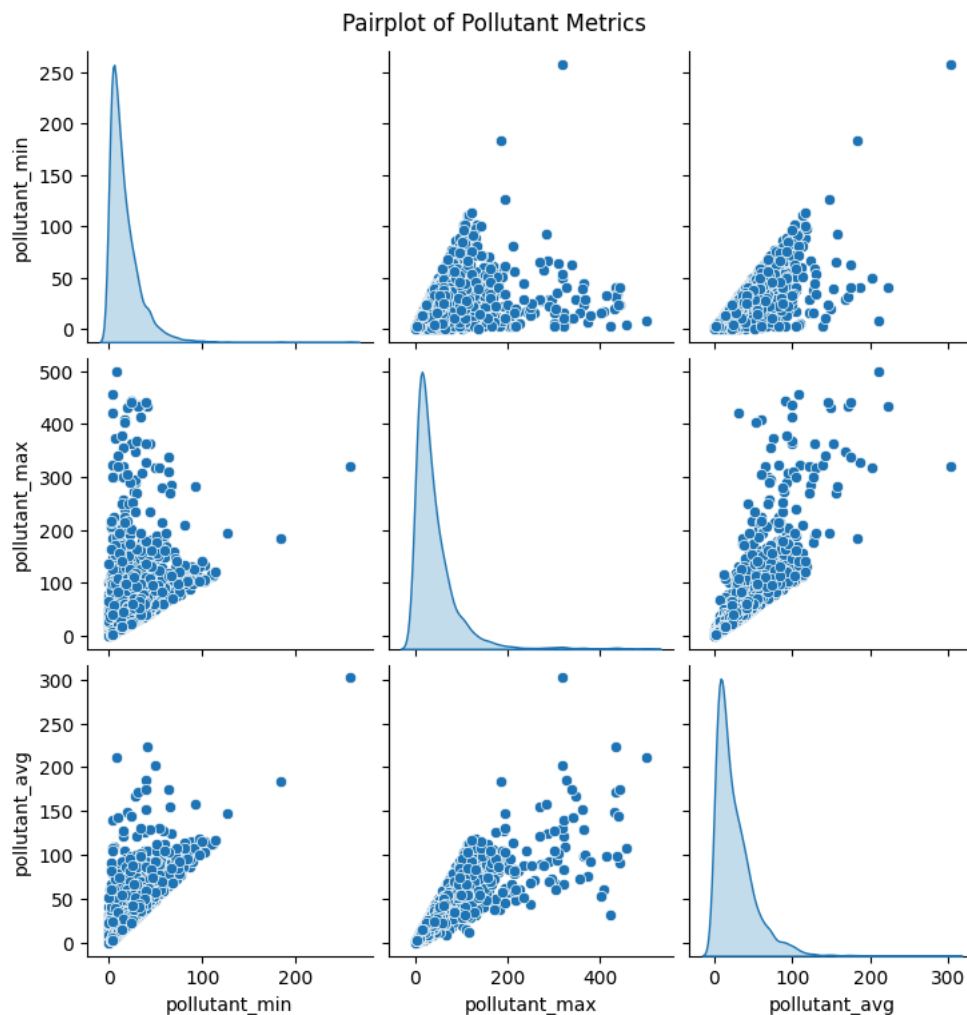


Average pollution is heavily influenced by maximum spikes.

Helps understand pollutant behavior and reliability of averages.

Provides evidence that monitoring extreme values (max) is critical for understanding overall air quality.

```
sns.pairplot(df[["pollutant_min","pollutant_max","pollutant_avg"]], diag_kind="kde")
plt.suptitle("Pairplot of Pollutant Metrics", y=1.02)
plt.show()
```



A pairplot shows scatterplots for every pair of numeric variables and distributions (KDE plots) along the diagonal.

You plotted three features: pollutant_min, pollutant_max, and pollutant_avg.

The diagonal plots show the distribution of each variable individually.

The off-diagonal plots show relationships between pairs of variables, showing both individual distributions and pairwise relationships in one chart.

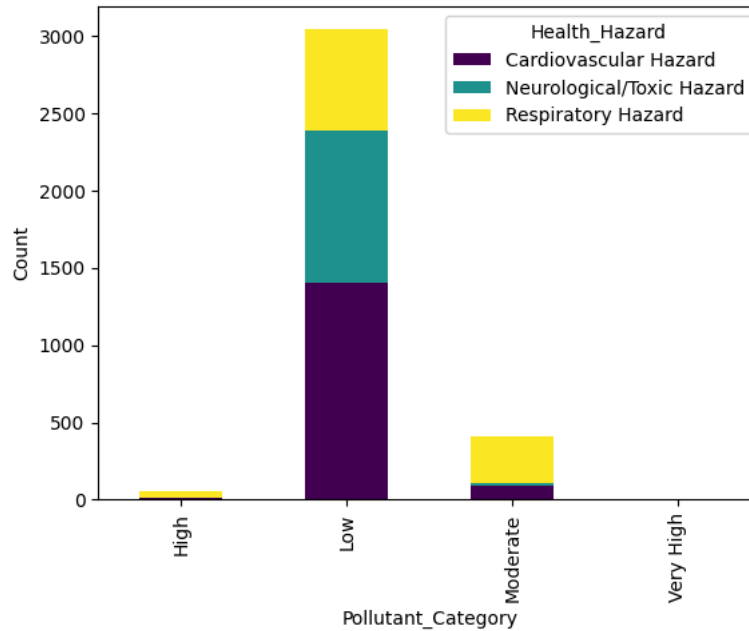
Confirms that pollutant metrics are strongly related — averages rise with peaks and minimums.

Helps detect outliers (points far from the main trend).

Provides a quick overview for feature selection in modeling: if variables are highly correlated, you may not need all of them in predictive models.

```
plt.figure(figsize=(10,6))
pd.crosstab(df["Pollutant_Category"], df["Health_Hazard"]).plot(kind="bar", stacked=True, colormap="viridis")
plt.title("Pollutant Category vs Health Hazard")
plt.ylabel("Count")
plt.show()
```

<Figure size 1000x600 with 0 Axes>



✓ Interpretation of your above chart

Relationship between pollution categories and associated health hazards.

Features used: Pollutant_Category, Health_Hazard.

Interpretation:

Low pollution is mostly linked to respiratory hazards.

High/Very High categories show more cardiovascular and toxic hazards.

This connects pollution severity directly to health risks, strengthening the business story for public awareness.

df.head()

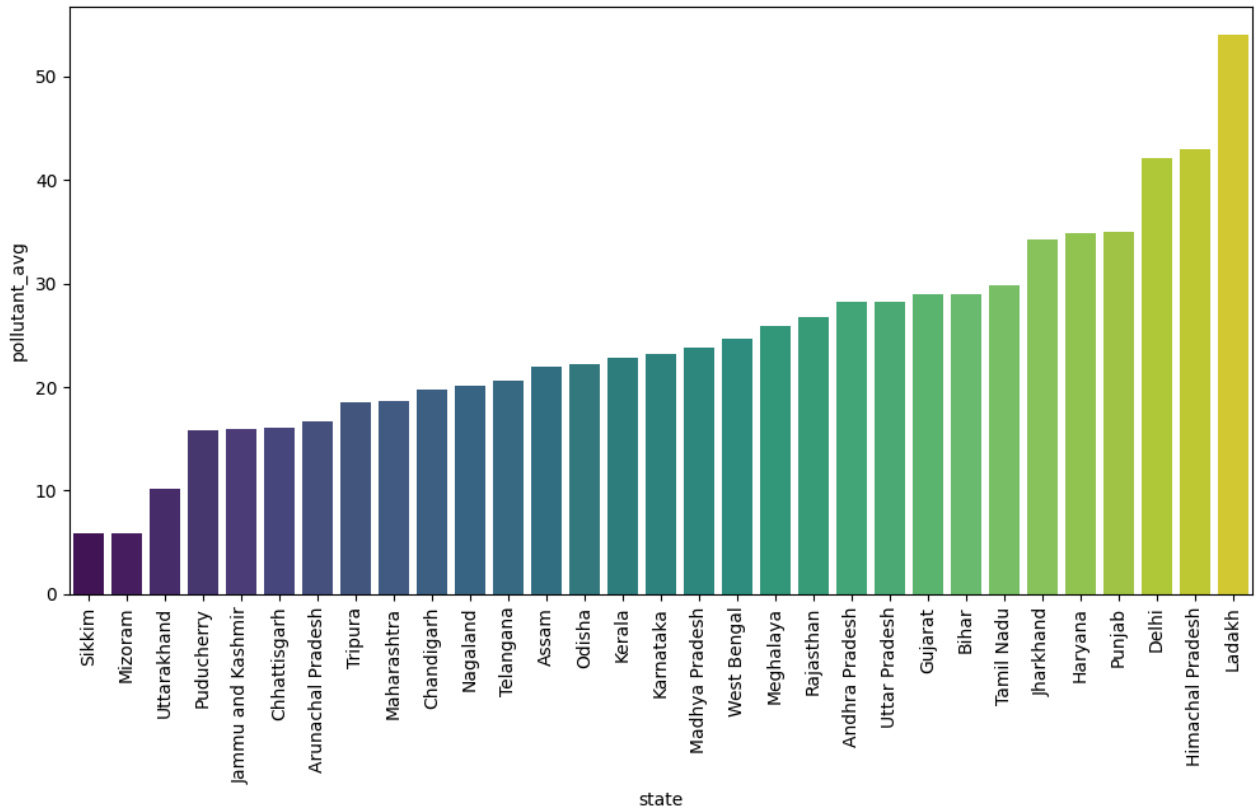
	country	state	city	station	latitude	longitude	Pollutant_Name	pollutant_min	pollutant_max	polluta
0	India	Andhra Pradesh	Kadapa	Yerramukkapalli, Kadapa - APPCB	14.465052	78.824187	OZONE	35.0	37.0	
1	India	Andhra Pradesh	Kakinada	Rama Rao Peta, Kakinada - APPCB	16.962488	82.229513	NO2	23.0	64.0	
2	India	Andhra Pradesh	Kakinada	Rama Rao Peta, Kakinada - APPCB	16.962488	82.229513	OZONE	16.0	30.0	
3	India	Andhra Pradesh	Machilipatnam	Srinivas Nagar Colony, Machilipatnam - APPCB	16.186555	81.132190	SO2	4.0	21.0	
4	India	Andhra Pradesh	Nellore	Ambedkar Nagar, Nellore - APPCB	14.438164	79.966814	NO2	26.0	33.0	

```
state_Avg= df.groupby("state")["pollutant_avg"].mean().reset_index()
state_Avg = state_Avg.sort_values(by="pollutant_avg", ascending=True)
plt.figure(figsize=(12,6))
sns.barplot(data=state_Avg,x="state",y="pollutant_avg",palette="viridis")
plt.xticks(rotation=90)
plt.title("Average Pollutant levels by Avg")
plt.show()
```

```
/tmp/ipykernel_4192/818836921.py:4: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

```
sns.barplot(data=state_Avg,x="state",y="pollutant_avg",palette="viridis")
```



The chart compares the average pollutant levels across all states, giving a clear picture of which states experience higher or lower air quality.

States with taller bars indicate higher average pollution, suggesting greater exposure risks for residents. States like Delhi, Punjab, Haryana, Bihar show higher averages → poor air quality.

States like Sikkim, Mizoram, Uttarakhand show lower averages → cleaner air.

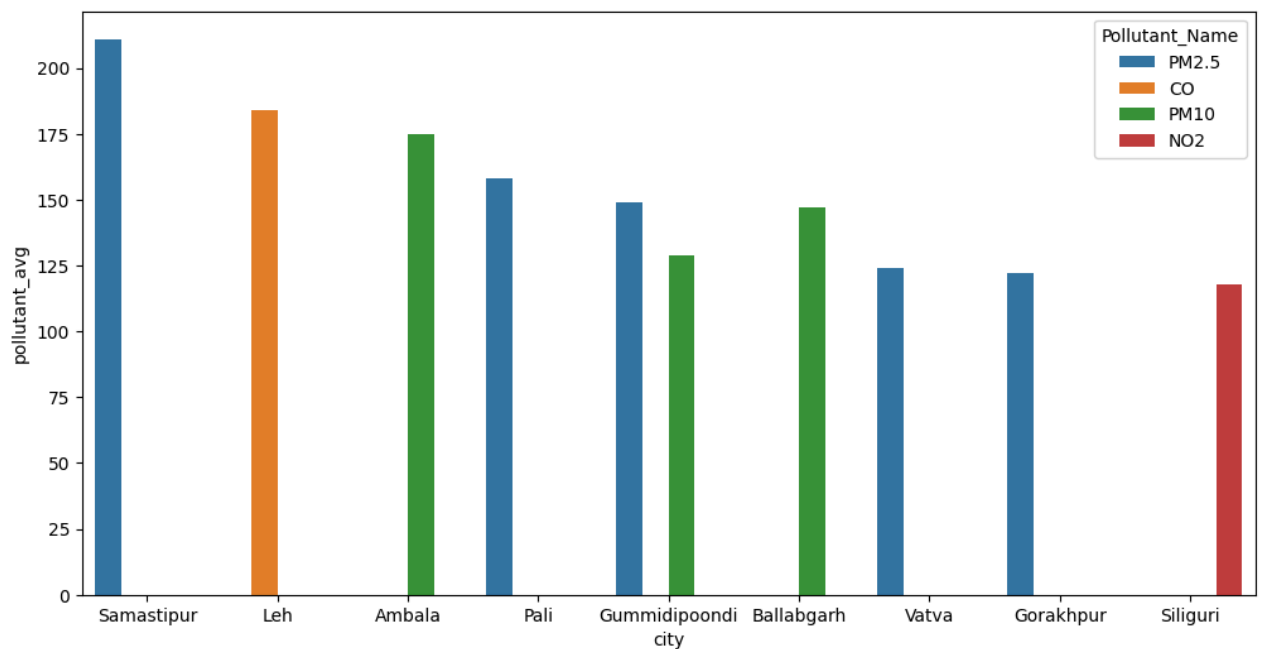
States on the left (shorter, darker bars) have the cleanest air.

States on the right (taller, brighter bars) have the highest average pollution.

The use of the viridis color palette helps visually distinguish states while keeping the focus on pollutant intensity.

This ordered view makes it easy to identify priority states for intervention and compare relative air quality across the country. Overall, the chart suggests that pollution is not evenly distributed — some states consistently face worse air quality, making them priority areas for environmental and health interventions.

```
City_pollutant=df.groupby(["city","Pollutant_Name"])["pollutant_avg"].mean().nlargest(10).reset_index()
plt.figure(figsize=(12,6))
sns.barplot(data=City_pollutant,x="city",y="pollutant_avg",hue="Pollutant_Name")
plt.title("Top 10 largest city_ Pollutant Average")
plt.show()
```



What the chart shows:

A bar chart of the 10 cities with the highest average pollutant levels, broken down by pollutant type (PM2.5, PM10, NO₂, SO₂, CO, O₃, NH₃).

Features used: city, Pollutant_Name, pollutant_avg.

Interpretation:

Cities like Samastipur, Lehigh, Ambala, Pali, Ballabgarh, Gorakhpur appear among the top polluted.

Samastipur has the highest average pollutant level (~210, dominated by PM2.5), showing severe particulate matter pollution.

Other cities show different dominant pollutants:

Leh → PM10, linked to dust and construction.

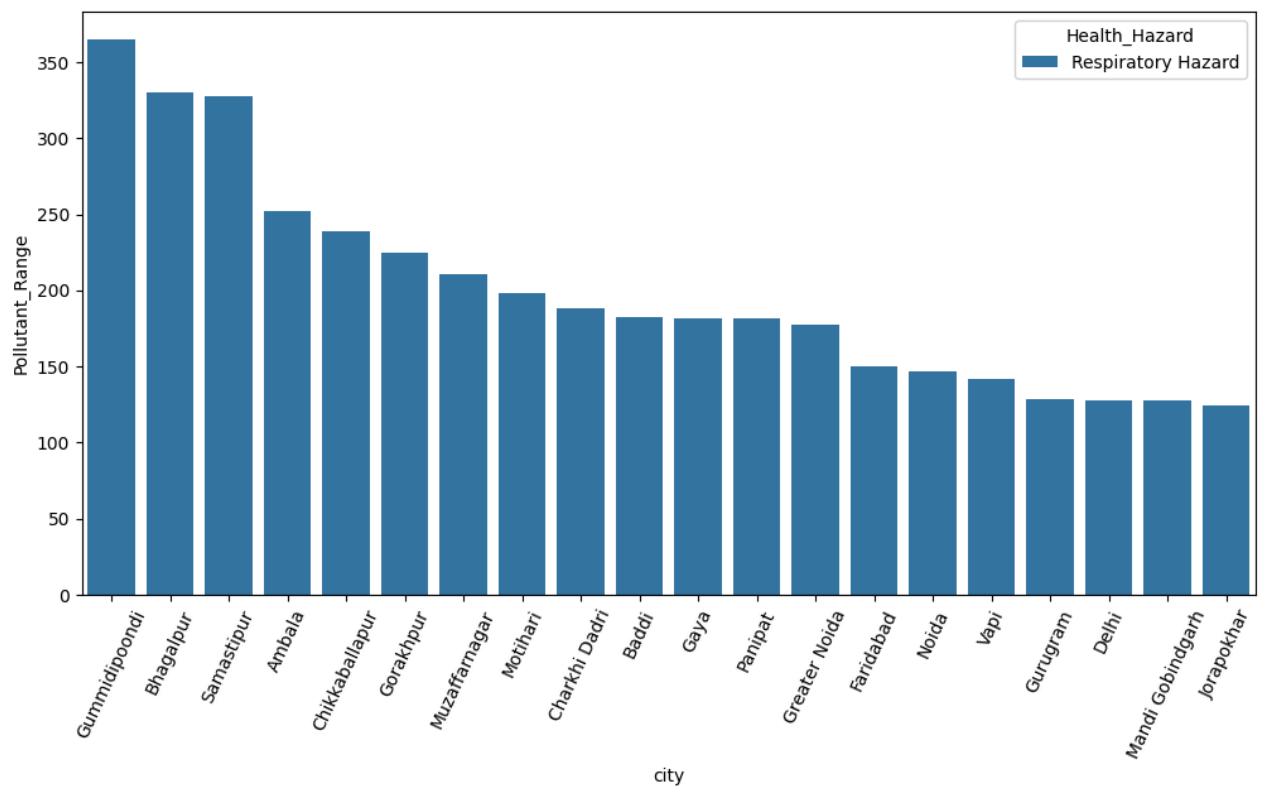
Ambala → NO₂, linked to vehicular emissions.

Ballabgarh → CO, linked to industrial and traffic sources.

This chart highlights that while all these cities face poor air quality, the pollutant mix differs city by city.

Policymakers can use this to design targeted interventions: stricter vehicle checks in Ambala, dust control in Leh, industrial emission regulation in Ballabgarh.

```
Top20_Health_hazard_cities = df.groupby(["city", "Health_Hazard"])["Pollutant_Range"].mean().nlargest(20).reset_index()
plt.figure(figsize=(12,6))
sns.barplot(data=Top20_Health_hazard_cities, x="city", y="Pollutant_Range", hue="Health_Hazard")
plt.xticks(rotation=65)
plt.title("Top20 Cities Health Hazard Distribution")
plt.show()
```



The chart highlights the top 20 cities with the highest pollutant ranges, associated with health hazards.

It shows that Respiratory issues dominate across these cities, suggesting pollutants like PM2.5 and PM10 are the primary drivers of health risks.

Other hazards (Cardiac, Neurological, etc.) appear less frequently, indicating they are either less strongly linked to pollution not as prevalent in the top polluted cities.

Overall, the chart suggests that air pollution in the most affected cities is strongly correlated with respiratory health problems, making them the most critical concern for public health interventions.

Steps to Download your cleaned data for dashboard creation using Power BI

```
#import pandas as pd
df.to_csv('AQI_Cleaned_Data.csv')
```

Stage 4 – Documentation, Insights and Presentation

Show a dashboard aligning all the charts in powerbi by connecting python in PowerBI

Summarize findings in plain English (what do the patterns mean)

Highlight 3–5 key insights (trends, anomalies, correlations)

Provide recommendations for business or decision-making

Explain the FINAL STORY WITH THE DASHBOARD/report

Create a PDF pasting your DASHBOARD, explaining all the above mentioned concepts and submit it

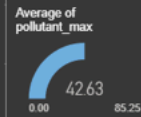
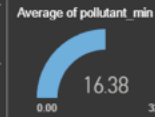
Air Quality Index - City based Pollutant Level and Distribution

Total Cities
3.5140K

Average Pollutant Level
26.59

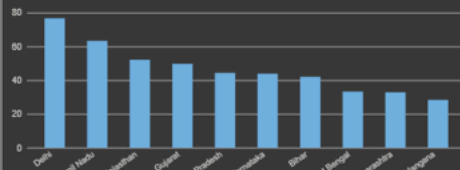
Lowest Pollutant Value
0

Highest Pollutant Value
500



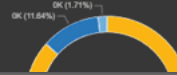
Average of Max pollutant_Level by State

Average of pollutant_max by state



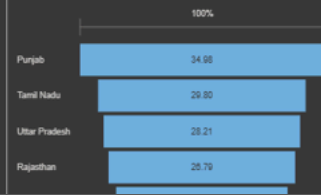
Pollutant category

category Count by Pollutant_Category



TOP 10 states with Average of pollutant

Average of pollutant_avg by state



Air Quality Index - Pollutant level and Health Hazard Distribution

Health_Hazard

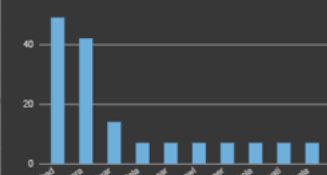
- ☐ Cardiovascular Hazard
- ☐ Neurological/Toxic H...
- ☐ Respiratory Hazard

Pollutant_Category

- ☐ High
- ☐ Low

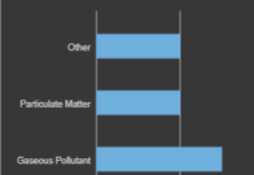
Pollutant Count - Bottom 10 Cities

Pollutant Group Count by city



Pollutant Group Count

by Pollutant_Group



Distribution of Hazard hazard

by Health_Hazard

